Python Exception Handling

Practice + Reference Answers

A Friendly Note Before We Begin

Exception handling is NOT magic. It is simply a way of saying:

> "I know something might go wrong here, and I am ready for it."

As a beginner, your job is NOT to master it today. Your job is to:

- Recognize exception handling
- Understand why it is used
- Read such programs without fear

That's it. No pressure

Basic Structure (Just Remember This Shape)

```
try:
    # risky code
except:
    # runs if error happens
else:
    # runs if no error
finally:
    # always runs
```

You will rarely use all four together as a beginner.

Question 1: Division Calculator

Story: User enters numbers. One day, user enters 0. Your program should not crash.

Reference Answer:

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
```

```
    result = a / b
    print("Result:", result)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Please enter valid numbers")
```

Why this works:

- Risky lines are inside `try`
- Specific errors are handled
- Program stays alive

Question 2: Invalid Input Handling

Story: User types text when number is expected.

Reference Answer:

```
try:
    age = int(input("Enter your age: "))
    print("Your age is:", age)
except ValueError:
    print("Age must be a number")
```

Important Lesson: Users are unpredictable. Your program must be patient.

Question 3: File Not Found

Story: Boss says: "Open data.txt" File does not exist.

Reference Answer:

```
try:
    f = open("data.txt", "r")
    print(f.read())
    f.close()
except FileNotFoundError:
    print("File not found. Please check file name.")
```

Real life: Missing files happen all the time. Programs should not panic.

Question 4: Writing to a File

Reference Answer:

```
try:
    f = open("output.txt", "w")
    f.write("Hello World")
    f.close()
except PermissionError:
    print("You do not have permission to write this file")
```

Question 5: Handling Multiple Errors Together

Reference Answer:

```
try:
    x = int(input("Enter number: "))
    y = int(input("Enter divisor: "))
    print(x / y)
except (ValueError, ZeroDivisionError):
    print("Invalid input or division by zero")
```

Tip: Multiple exceptions can be grouped.

Question 6: Using else and finally

Story: Bank Withdrawal

Reference Answer:

```
try:
    balance = int(input("Enter balance: "))
    withdraw = int(input("Enter withdrawal amount: "))
    balance = balance - withdraw
except ValueError:
    print("Invalid input")
else:
    print("Remaining balance:", balance)
finally:
    print("Thank you for using the service")
```

Key Idea:

- `else` runs only if no error

- `finally` always runs

Question 7: Raising Your Own Exception

Story: Signup system. Age must be 18 or above.

Reference Answer:

```
try:
    age = int(input("Enter age: "))
    if age < 18:
        raise ValueError("Age below 18 not allowed")
    print("Signup successful")
except ValueError as e:
    print(e)
```

Why raise exceptions? When YOU know something is wrong, even if Python doesn't.

Question 8: Mini Real-World Example

Online Order System

```
try:
    qty = int(input("Enter quantity: "))
    price = 100
    total = qty * price
    print("Total amount:", total)
except ValueError:
    print("Quantity must be a number")
finally:
    print("Order process completed")
```

Final Words

- Errors are normal
- Exception handling is protection, not decoration
- You are NOT expected to write perfect exception handling now
- Just learn to read and understand it

That's enough for your first class on exceptions, you will later use it